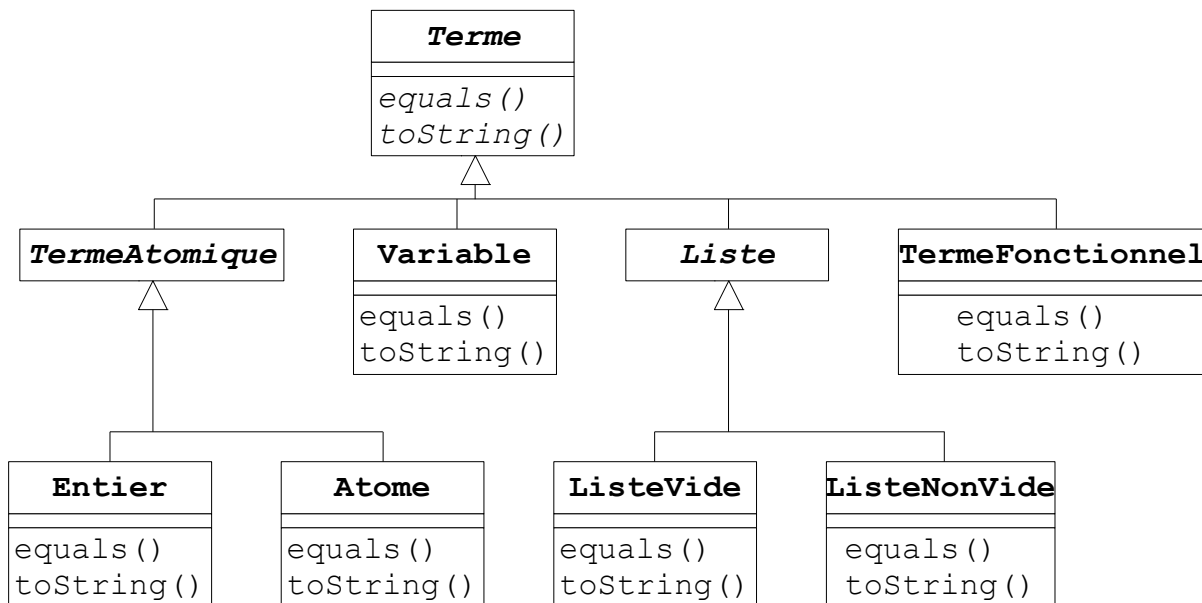


Prolog

Un programme écrit en langage Prolog est constitué de *termes*. Ces termes peuvent être de différentes sortes. Nous considérons cinq sortes différentes de *termes* : les *entiers*, les *atomes*, les *variables* et les *listes*. Nous nous proposons de définir des classes permettant de créer des instances de ces différents *termes*. Le diagramme des classes est donné ci-dessous.



Question 1

Définir les constructeurs et méthodes de la classe `Entier`. Un entier ne peut être égal qu'à un entier de même valeur, ou à une variable dont la valeur est un entier de même valeur.

```

public class Entier extends TermeAtomique {
    private long valeur;
    public Entier(long valeur){...}
    public String toString(){...}
    public boolean equals(Object obj) {...}
}
  
```

Question 2

Définir les constructeurs et méthodes de la classe `Atome`. Un atome ne peut être égal qu'à un atome de même valeur, ou à une variable dont la valeur est un atome de même valeur.

```

public class Atome extends TermeAtomique {
    private String valeur;
    public Atome (String valeur) {...}
    public String toString(){...}
    public boolean equals(Object obj) {...}
}
  
```

Question 3

Définir les constructeurs et méthodes de la classe `Variable`. Une variable peut ne pas avoir de valeur (l'attribut `valeur` vaut `null`), ou avoir une valeur (l'attribut `valeur` est différent de `null`). La méthode `toString` retourne une chaîne de caractères constituée du

nom de la variable suivi de "=", suivi de "?" si la variable n'a pas de valeur, ou suivi de la valeur de la variable convertie en chaîne de caractères si la variable a une valeur.

```
public class Variable extends Terme {
    private String nom;
    private Terme valeur;
    public Variable(String nom) {...}
    public Variable(String nom, Terme valeur){...}
    public Terme getValeur(){...}
    public String toString(){...}
    public boolean equals(Object obj){...}
}
```

Question 4

Définir les constructeurs et méthodes de la classe `ListeVide`. La méthode `toString` retourne la chaîne de caractères "[]".

```
public class ListeVide extends Liste {
    public ListeVide(){...}
    public String toString(){...}
    public boolean equals(Object obj){...}
}
```

Question 5

Définir les constructeurs et méthodes de la classe `ListeNonVide`.

Le constructeur ayant une variable en deuxième argument, n'accepte comme

```
public class ListeNonVide extends Liste {
    private Terme tete;
    private Terme queue;
    public ListeNonVide(Terme tete, Liste queue){...}
    public ListeNonVide(Terme tete, Variable queue){...}
    public String toString(){...}
    public boolean equals(Object obj){...}
}
```

deuxième argument qu'une variable qui n'a pas de valeur, ou une variable dont la valeur est une liste, qui est la queue de la liste. On écrira 2 définitions de la méthode `toString` :

1. La méthode `toString` retourne la chaîne de caractères "[" + la tête de la liste convertie en chaîne de caractères + "|" + la queue de la liste convertie en chaîne de caractères + "]" Exemple : [1|[2|[3|[]]]]
2. La méthode `toString` retourne une chaîne de caractères constituée de la liste des éléments de la liste séparés par des ',' et entre crochets. Exemple : [1, 2, 3]

Question 6

Définir le constructeur et les méthodes de la classe `TermeFonctionnel`.

```
public class TermeFonctionnel extends Terme {
    private String foncteur;
    private Liste parametres;
    public TermeFonctionnel(String f, Liste p){...}
    public String toString(){...}
    public boolean equals(Object obj){...}
}
```

Question 7

Que faire pour que les chaînes de caractères obtenues par les différentes méthodes `toString()` soient précédées du nom de la classe suivi de ':' .